

MLFQ Scheduler Design Document

Project: Multi-Level Feedback Queue Scheduler for xv6-RISC-V

Team Members: Saad Inam - 29068, Hassan Jabbar - 29060

1. Overview

This project implements a Multi-Level Feedback Queue (MLFQ) scheduler to replace xv6's default round-robin scheduler. The MLFQ scheduler provides priority-based scheduling with dynamic process priority adjustment based on CPU usage patterns.

2. Design Decisions

2.1 Queue Structure

- **Number of queues:** 4 priority levels (0-3)
 - Level 0: Highest priority
 - Level 3: Lowest priority
- **Implementation:** Linked lists using `next_proc` pointer in `struct proc`. No array shifting needed.

2.2 Time Quanta

Each priority level has a different time quantum:

Level 0: 4 ticks (highest priority, shortest quantum)

Level 1: 8 ticks

Level 2: 16 ticks

Level 3: 32 ticks (lowest priority, longest quantum)

Rationale:

- Short quanta for high priority → better responsiveness
- Long quanta for low priority → less context switching for CPU-bound processes
- Exponential growth balances fairness

2.3 Process Lifecycle

New Process

- Starts at queue level 0
- Gets initial quantum of 4 ticks
- `ticks_used` initialized to 0

Running Process

- Timer interrupt increments `ticks_used` every tick
- When yielding CPU:
 - If `ticks_used >= quantum`: demote to next lower level
 - Otherwise: re-queue at current level
- `ticks_used` resets when entering new queue

I/O-Blocked Process

- When waking from sleep: re-queued at **current level** (no demotion)
- Preserves high priority for interactive processes

Demotion

- Process demoted when it uses full quantum
- Cannot demote below level 3
- Quantum updated to new level's value

2.4 Starvation Prevention

Priority Boosting:

- Every 200 ticks, all `RUNNABLE`/`RUNNING` processes boosted to level 0
 - Ensures low-priority processes eventually get CPU time
 - Prevents indefinite starvation
-

3. Data Structures

3.1 Modified struct proc (kernel/proc.h)

```
struct proc {  
    // ... existing fields ...  
  
    // MLFQ Scheduler fields  
    int queue_level;           // Current queue level (0-3)  
    int ticks_used;           // Ticks consumed in current queue  
    struct proc *next_proc;    // Linked list pointer for queuing  
    int quantum;              // Time quantum for current queue level  
};
```

3.2 MLFQ Queue Structure (kernel/proc.c)

```
struct {  
    struct proc *head; // Head of queue at this level  
    struct proc *tail; // Tail of queue at this level  
} mlfq[MLFQ_LEVELS]; // 4 queues, one per level
```

3.3 Global Counter

```
static uint boost_counter = 0; // Tracks ticks until next boost
```

4. Key Functions

4.1 Queue Management

- `get_quantum(level)` - Returns quantum for given level
- `mlfq_enqueue(p, level)` - Add process to queue (FIFO)
- `mlfq_dequeue(level)` - Remove first process from queue
- `mlfq_remove(p)` - Remove process from its current queue
- `mlfq_next()` - Get next runnable process (highest priority first)

4.2 Scheduling Logic

- `mlfq_demote(p)` - Demote process to next lower level
- `mlfq_boost()` - Boost all processes to level 0
- `scheduler()` - Main scheduler loop (uses `mlfq_next()`)
- `yield()` - Check for demotion, re-queue process

4.3 System Call

- `getprocinfo(pid, struct procinfo*)` - Returns process MLFQ state
 - Used for testing and debugging
 - Returns: pid, queue_level, ticks_used, quantum, name, state
-

5. Scheduling Algorithm

Pseudocode for `scheduler()`:

```
while true:
    enable interrupts
    disable interrupts

    p = mlfq_next() // Get from highest priority non-empty queue
    if p != NULL:
        p.state = RUNNING
        context_switch(scheduler_context, p.context)
        release(p.lock)
    else:
        wait_for_interrupt()
```

Pseudocode for `yield()`:

```
acquire(p.lock)

if p.ticks_used >= p.quantum:
    mlfq_demote(p) // Move to lower queue

p.state = RUNNABLE
mlfq_enqueue(p, p.queue_level)
```

```
context_switch(p.context, scheduler_context)
release(p.lock)
```

Pseudocode for timer interrupt (trap.c):

```
p.ticks_used++
boost_counter++
```

```
if boost_counter >= BOOST_INTERVAL:
    boost_counter = 0
    mlfq_boost()
```

```
yield()
```

6. Testing Plan

Test Cases:

1. **Single CPU-bound process**
 - Should gradually demote to level 3
 - Should get boosted every 200 ticks
2. **Single I/O-bound process**
 - Should remain at level 0
 - Should complete quickly
3. **Mixed workload**
 - CPU-bound process demotes
 - I/O-bound process stays high priority
 - Both eventually complete
4. **Starvation test**
 - Multiple CPU-bound processes
 - Verify all make progress via boosting

Validation:

- Use `getprocinfo()` to check queue levels
 - Monitor `ticks_used` and `quantum` values
 - Verify demotion after quantum expiration
 - Confirm boosting resets all to level 0
-

7. Challenges & Solutions

Challenge 1: Lock Ordering

Issue: Deadlock risk when accessing queues and process locks

Solution: `mlfq_next()` acquires process lock before removing from queue

Challenge 2: Running Process State

Issue: Running process not in any queue during boost

Solution: `mlfq_boost()` checks for RUNNING state and updates fields without enqueueing

Challenge 3: I/O Priority

Issue: I/O-bound processes shouldn't be demoted

Solution: `wakeup()` re-queues at current level (no demotion)

8. Expected Behavior

CPU-bound Process Timeline:

Time	Queue	Ticks	Action
0	0	0	Start
4	1	0	Demote (used quantum)
12	2	0	Demote (used quantum)
28	3	0	Demote (used quantum)
60	3	0	Stay at level 3
200	0	0	Boosted

I/O-bound Process Timeline:

Time	Queue	Ticks	Action
0	0	0	Start
2	0	2	I/O block
4	0	2	Resume (no demotion)
6	0	0	I/O block
8	0	0	Resume

9. Syscall Screenshot

```
saad@Saads-MacBook-Air xv6-labs-2025 % make qemu
init: starting sh
$ proctest
Current process info:
PID: 3
Queue Level: 0
Ticks Used: 0
Quantum: 4
Name: proctest
$ mlfqtest
=== MLFQ Scheduler Test ===
Testing with CPU-bound and I/O-bound processes

CPU-I/O-bound process u[nd] process [PID 6]P ID 5] st[art]ing
ting
I/O [PID 6]: queue=0, ticks=0, quantum=4
CPU [PID 5]: queue=0, ticks=3, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
CPU [PID 5]: queue=1, ticks=1, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
CPU [PID 5]: queue=1, ticks=3, quantum=8
I/O [PID 6]: queue=0, ticks=0, quantum=4
CPU [PID 5]: queue=1, ticks=6, quantum=8
I/O [PID 6]: queue=0, ticks=0, quantum=4
CPU [PID 5]: queue=2, ticks=0, quantum=16
CPU-bound process [PID 5] finished
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O [PID 6]: queue=0, ticks=0, quantum=4
I/O-bound process [PID 6] finished

=== Test Complete ===
Expected behavior:
- CPU-bound process should move to lower queues (0->1->2->3)
- I/O-bound process should stay at high priority (queue 0)
- Both should eventually complete due to boosting
$
```